



Cypher is Turing-Complete

A Formal Proof via 2-Counter Machine Simulation

Pierre Halftermeyer · Neo4j · March 2026

Abstract. We prove that Cypher 25 is Turing-complete by showing that `reduce()`, `CASE`, and list comprehensions can simulate any 2-counter machine (Minsky 1967). We address the bounded-step objection and verify the construction on Neo4j Aura.

§1 The Claim

Cypher 25, restricted to a single `RETURN` with `reduce()`, list comprehensions, and `CASE`, is **Turing-complete**.

Proof strategy. By reduction: $\text{TM} \xrightarrow{\text{Minsky}} \text{2CM} \xrightarrow{\text{this}} \text{Cypher}$.

§2 2-Counter Machines

A **2-counter machine** $M = (Q, q_0, q_h, \delta)$ has a finite state set Q , initial state q_0 , halt state q_h , and transition function $\delta : Q \rightarrow \text{Instr}$ where each instruction is:

- $\text{INC}(c, q')$ — increment $c \in \{A, B\}$, goto q'
- $\text{JZDEC}(c, q_z, q_p)$ — if $c=0$ goto q_z , else decrement and goto q_p

Fact (Minsky 1967)

For any TM T , $\exists$ a 2-counter machine simulating T .

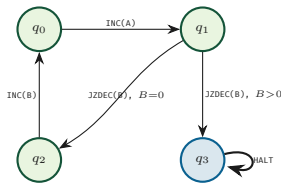


Fig. 1 — Example 4-state program.

§3 Encoding in Cypher

Program. Encode δ as a list of maps:

```

LET program = [
  {state:0, op:'INC', counter:'A', next:1},
  {state:1, op:'JZDEC', counter:'B',
    q_zero:2, q_pos:3},
  {state:2, op:'INC', counter:'B', next:0},
  {state:3, op:'HALT', counter:'', next:3}
]

```

State. Configuration (q, c_A, c_B) as map $\{\text{state}:0, A:0, B:0\}$. Convention: $\text{state}=-1$ means halted.

§4 The Simulation

Each `reduce()` iteration executes one machine step. The `head([v IN [e] ...])` idiom provides let-binding inside `reduce()` (where `LET` is unavailable).

```

CYPHER 25
LET program = [ /* ... */ ]
LET max_steps = 1000000

LET result = reduce(
  machine = {state:0, A:0, B:0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
    THEN machine
  ELSE
    head([instr IN [program[machine.state]] |
      CASE instr.op
        WHEN 'INC' THEN
          CASE instr.counter
            WHEN 'A' THEN {state:instr.next,
              A: machine.A + 1, B: machine.B}
            WHEN 'B' THEN {state:instr.next,
              A: machine.A, B: machine.B + 1}
          END
        WHEN 'JZDEC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              CASE WHEN machine.A = 0
                THEN {state:instr.q_zero,
                  A: 0, B: machine.B}
              ELSE {state:instr.q_pos,
                  A: machine.A-1, B: machine.B}
              END
            WHEN 'B' THEN
              CASE WHEN machine.B = 0
                THEN {state:instr.q_zero,
                  A: machine.A, B: 0}
              ELSE {state:instr.q_pos,
                  A: machine.A, B: machine.B-1}
              END
          END
        WHEN 'HALT' THEN
          {state:-1, A: machine.A, B: machine.B}
      ])
    END
  )
RETURN result

```

§5 Correctness

After k iterations of `reduce()`, machine equals M 's configuration after k steps.

Base. $k=0$: machine = $\{0, 0, 0\} = M_0$. ✓

Step. Assume machine = (q_k, a_k, b_k) matches M .

At $k+1$, `reduce()` looks up $\delta(q_k)$ and executes:

$\text{INC}(A, q') \rightarrow (q', a_k+1, b_k)$ ✓

$\text{JZDEC}(A, ..), a_k=0 \rightarrow (q_z, 0, b_k)$ ✓

$\text{JZDEC}(A, ..), a_k>0 \rightarrow (q_p, a_k-1, b_k)$ ✓

$\text{HALT} \rightarrow (-1, ..)$, identity after ✓

Symmetric for B . □

§6 The Bounded-Step Objection

The `reduce()` uses `range(1, max_steps)` — a finite bound.

§6.1 Resolution via IN TRANSACTIONS

While the pure functional version stays within a single expression, the practical unbounded version uses graph storage for state persistence.

```
CYPHER 25
CREATE (:Machine {state:0, A:0, B:0});

UNWIND range(1, 9223372036854775807) AS step
CALL (step) {
  MATCH (m:Machine)
  WITH m,
    CASE WHEN m.state = -1 THEN 1/0
    ELSE $program[m.state]
  END AS instr
  SET m.state = CASE instr.op
  WHEN 'INC' THEN instr.next
  WHEN 'JZDEC' THEN CASE instr.counter
  WHEN 'A' THEN CASE WHEN m.A = 0
  THEN instr.q_zero ELSE instr.q_pos END
  WHEN 'B' THEN CASE WHEN m.B = 0
  THEN instr.q_zero ELSE instr.q_pos END
  END
  WHEN 'HALT' THEN -1 END,
  m.A = CASE
  WHEN instr.op='INC' AND instr.counter='A'
  THEN m.A+1
  WHEN instr.op='JZDEC' AND instr.counter='A'
  AND m.A > 0 THEN m.A-1
  ELSE m.A END,
  m.B = CASE
  WHEN instr.op='INC' AND instr.counter='B'
  THEN m.B+1
  WHEN instr.op='JZDEC' AND instr.counter='B'
  AND m.B > 0 THEN m.B-1
  ELSE m.B END
} IN TRANSACTIONS OF 1 ROW
ON ERROR BREAK
```

Termination. At halt, state becomes -1 . Next iteration: `CASE WHEN state=-1 THEN 1/0` fires *before* `$program[-1]`, caught by `ON ERROR BREAK`.

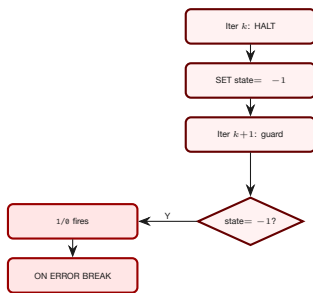


Fig. 2 — Termination via `1/0` + `ON ERROR BREAK`.

Note. $2^{63} \approx 9.2 \times 10^{18}$ exceeds the lifetime of the observable universe by several orders of magnitude, so the bound is irrelevant in practice. True theoretical unboundedness would need an external restart loop.

§6.2 Sufficiency Argument

For TM T halting in $f(|w|)$ steps: set `max_steps =  $f(|w|)$` . Cypher correctly decides every decidable language — **universal for total functions**.

§7 Required Primitives

Primitive	Role
<code>reduce()</code>	Iteration (fold)
<code>CASE WHEN</code>	Branching
<code>+1, -1, = 0</code>	Counter ops
<code>Map {s,A,B}</code>	State repr.
<code>prog[i]</code>	$O(1)$ lookup
<code>head([..])</code>	Let-binding

No graph operations. No APOC. No GDS.

§8 Verified on Neo4j Aura

Both approaches tested on Aura (Neo4j 5.x, Cypher 25). Test program: 4 instructions, expected $\{-1, 2, 0\}$.

Step	Instr	St	A	B
0	INC(A)	q_0	$0 \rightarrow 1$	0
1	JZDEC(B), $B=0$	q_1	1	0
2	INC(B)	q_2	1	$0 \rightarrow 1$
3	INC(A)	q_0	$1 \rightarrow 2$	1
4	JZDEC(B), $B>0$	q_1	2	$1 \rightarrow 0$
5	HALT	q_3	2	0

reduce() result

{A:2, B:0, state:-1} ✓

IN TRANSACTIONS result

m.state=-1, m.A=2, m.B=0 ✓

§9 Complete Runnable Queries

The following two queries are copy-paste ready for any Neo4j instance with Cypher 25 support. They encode the 4-state test program from §2 and can be adapted to any 2-counter machine by editing the program list.

§9.1 Approach 1: Pure reduce()

A single `RETURN` statement — no graph writes, no side effects. The `head([v IN [expr] | ...])` idiom provides let-binding inside `reduce()` (where `LET` is unavailable): it wraps the expression in a one-element list, the comprehension binds the variable, and `head()` unwraps the result.

```
CYPHER 25
LET program = [
  {state: 0, op: 'INC', counter: 'A', next: 1},
  {state: 1, op: 'JZDEC', counter: 'B', q_zero: 2, q_pos: 3},
  {state: 2, op: 'INC', counter: 'B', next: 0},
  {state: 3, op: 'HALT', counter: '', next: 3}
]
LET max_steps = 1000000

LET result = reduce(
  machine = {state: 0, A: 0, B: 0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
  THEN machine
  ELSE
    head([instr IN [program[machine.state]] |
      CASE instr.op
      WHEN 'INC' THEN
        CASE instr.counter
        WHEN 'A' THEN
          {state: instr.next,
```

```

        A: machine.A + 1, B: machine.B}
    WHEN 'B' THEN
        {state: instr.next,
        A: machine.A, B: machine.B + 1}
    END
    WHEN 'JZDEC' THEN
        CASE instr.counter
        WHEN 'A' THEN
            CASE WHEN machine.A = 0
            THEN {state: instr.q_zero,
            A: 0, B: machine.B}
            ELSE {state: instr.q_pos,
            A: machine.A - 1,
            B: machine.B}
            END
        WHEN 'B' THEN
            CASE WHEN machine.B = 0
            THEN {state: instr.q_zero,
            A: machine.A, B: 0}
            ELSE {state: instr.q_pos,
            A: machine.A,
            B: machine.B - 1}
            END
        END
    WHEN 'HALT' THEN
        {state: -1,
        A: machine.A, B: machine.B}
    END
    })
    END
    )
    RETURN result

```

§9.2 Approach 2: IN TRANSACTIONS

Uses graph storage for state persistence. The 1/0 guard fires *before* any list indexing when halted, caught by ON ERROR BREAK.

```

CYPHER 25
CREATE (:Machine {state: 0, A: 0, B: 0});

```

```

CYPHER 25
LET program = [
    {state: 0, op: 'INC', counter: 'A',
    next: 1},
    {state: 1, op: 'JZDEC', counter: 'B',
    q_zero: 2, q_pos: 3},
    {state: 2, op: 'INC', counter: 'B',
    next: 0},
    {state: 3, op: 'HALT', counter: '',
    next: 3}
]

UNWIND range(1, 9223372036854775807) AS step
CALL (step) {
    MATCH (m:Machine)
    WITH m,
        CASE WHEN m.state = -1 THEN 1/0
        ELSE program[m.state]
        END AS instr
    SET m.state = CASE instr.op
    WHEN 'INC' THEN instr.next
    WHEN 'JZDEC' THEN
        CASE instr.counter
        WHEN 'A' THEN
            CASE WHEN m.A = 0
            THEN instr.q_zero
            ELSE instr.q_pos END
        WHEN 'B' THEN
            CASE WHEN m.B = 0
            THEN instr.q_zero
            ELSE instr.q_pos END
        END
    WHEN 'HALT' THEN -1
    END,
    m.A = CASE
    WHEN instr.op = 'INC'
    AND instr.counter = 'A'
    THEN m.A + 1
    WHEN instr.op = 'JZDEC'
    AND instr.counter = 'A'
    AND m.A > 0 THEN m.A - 1
    ELSE m.A END,
    m.B = CASE

```

```

    WHEN instr.op = 'INC'
    AND instr.counter = 'B'
    THEN m.B + 1
    WHEN instr.op = 'JZDEC'
    AND instr.counter = 'B'
    AND m.B > 0 THEN m.B - 1
    ELSE m.B END
} IN TRANSACTIONS OF 1 ROW
ON ERROR BREAK

```

```

// Read result:
MATCH (m:Machine) RETURN m;

```

§9.3 Approach 3: Graph-Native QPP Traversal

The most idiomatic approach: model the machine as a *graph* and let pattern matching *be* the computation. JZDEC splits into two edges (JZDEC_ZERO, JZDEC_POS); the predicate in allReduce prunes the invalid branch at traversal time.

```

CYPHER 25
// Build the machine as a graph
CREATE (q0:State:Init {id: 0})
CREATE (q1:State {id: 1})
CREATE (q2:State {id: 2})
CREATE (q3:State:Halt {id: 3})
CREATE (q0)-[:INC {c:'A'}]->(q1)
CREATE (q1)-[:JZDEC_ZERO {c:'B'}]->(q2)
CREATE (q1)-[:JZDEC_POS {c:'B'}]->(q3)
CREATE (q2)-[:INC {c:'B'}]->(q0)

```

```

CYPHER 25
// Traverse: allReduce prunes, NEXT extracts
MATCH REPEATABLE ELEMENTS
p = (init:Init)
((()-[t:INC|JZDEC_ZERO|JZDEC_POS]->())
{1, 9223372036854775807}
(h:Halt)
WHERE allReduce(
    m = {A: 0, B: 0}, t |
    CASE
        WHEN t:INC AND t.c = 'A'
        THEN {A: m.A+1, B: m.B}
        WHEN t:INC AND t.c = 'B'
        THEN {A: m.A, B: m.B+1}
        WHEN t:JZDEC_POS AND t.c = 'A'
        THEN {A: m.A-1, B: m.B}
        WHEN t:JZDEC_POS AND t.c = 'B'
        THEN {A: m.A, B: m.B-1}
        ELSE m
    END,
    CASE
        WHEN t:JZDEC_ZERO AND t.c = 'A'
        THEN m.A = 0
        WHEN t:JZDEC_ZERO AND t.c = 'B'
        THEN m.B = 0
        WHEN t:JZDEC_POS AND t.c = 'A'
        THEN m.A > 0
        WHEN t:JZDEC_POS AND t.c = 'B'
        THEN m.B > 0
        ELSE true
    END
)
RETURN p
NEXT
// Re-derive counters from the valid path
RETURN length(p) AS steps,
    reduce(m = {A:0, B:0},
    t IN relationships(p) |
    CASE
        WHEN t:INC AND t.c = 'A'
        THEN {A: m.A+1, B: m.B}
        WHEN t:INC AND t.c = 'B'
        THEN {A: m.A, B: m.B+1}
        WHEN t:JZDEC_POS AND t.c = 'A'
        THEN {A: m.A-1, B: m.B}
        WHEN t:JZDEC_POS AND t.c = 'B'
        THEN {A: m.A, B: m.B-1}
        ELSE m
    END

```

```
) AS result
```

Key properties:

- The machine *is* the graph; computation *is* path finding
- REPEATABLE ELEMENTS allows revisiting states (loops)
- allReduce prunes eagerly: once a JZDEC branch fails the predicate, that path is abandoned immediately
- No HALT self-loop needed — reaching a :Halt node terminates the pattern
- reduce() in the NEXT stage re-derives final counter values from the surviving path (same pattern as [3])

References

- [1] Minsky, M. L. (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall.
- [2] Neo4j: [Cypher Manual](#).
- [3] Halftermeyer, P. (2025). Declarative Route Planning for EVs With Neo4j — Graph Traversal Grows Up. [Neo4j Developer Blog](#).
- [4] Halftermeyer, P. (2015–2025). Advent of Code in Pure Cypher — 150+ puzzles solved purely in Cypher, empirical evidence of expressiveness. github.com/halftermeyer/adventofcode